MATERI PRAKTIKUM PENGAYAAN: PEMBAHASAN MENDALAM DATABASE VIEW & STUDI KASUS

Modul pengayaan ini disusun secara komprehensif untuk melengkapi Materi Praktikum 3 (SELECT, VIEW, CONTROL FLOW FUNCTION) pada mata kuliah Sistem Manajemen Basis Data Lanjut (SMBD Lanjut). Dokumen ini mengupas tuntas arsitektur View, aturan manipulasi data (Updatable Views), proteksi integritas transaksi dengan klausa WITH CHECK OPTION, algoritma pemrosesan internal MySQL (MERGE vs TEMPTABLE), mekanisme keamanan hak akses (DEFINER vs INVOKER), serta menyediakan 10 latihan soal berbasis studi kasus transaksi penjualan minimarket (adaptasi normalisasi 3NF Diskusi Kelompok 7) lengkap dengan kunci jawaban terpisah bagi Dosen dan Asisten Praktikum.

1. KONSEP DASAR & ARSITEKTUR DATABASE VIEW

Dalam sistem basis data relasional (RDBMS), View adalah tabel virtual (logical table) yang strukturnya didefinisikan berdasarkan himpunan hasil kueri SQL (result set). Berbeda dengan tabel fisik (base table), View TIDAK menyimpan data fisik tersendiri di dalam storage engine. Yang tersimpan secara permanen di dalam data dictionary hanyalah metadata dan kueri SQL pendefinisiannya. Saat View dipanggil melalui klausa SELECT, DBMS akan mengeksekusi kueri pembentuknya secara dinamis.

1.1 Keuntungan dan Alasan Utama Penggunaan View

- **Keamanan dan Masking Data (Security & Data Masking):** Memungkinkan penyembunyian baris atau kolom sensitif dari pengguna tertentu tanpa perlu merubah skema fisik. Contoh: Akun front-office kasir hanya diberikan akses ke View yang menyembunyikan kolom password kasir atau batas kredit customer.
- **Penyederhanaan Kueri Kompleks (Query Simplification):** Kueri kompleks yang melibatkan multi-tabel JOIN, pengelompokan data, dan kalkulasi matematis dapat disederhanakan menjadi kueri sederhana: 'SELECT * FROM nama_view'. Hal ini mempermudah integrasi sistem bagi pengembang aplikasi klien.
- **Sentralisasi dan Konsistensi Logika Bisnis:** Menjamin seluruh departemen atau aplikasi klien menggunakan formula bisnis yang seragam (misal formula total nota = banyak * hargajual, atau penentuan klasifikasi loyalitas pelanggan). Perubahan formula di masa depan cukup dilakukan pada satu View terpusat di basis data.
- **Independensi Data Logis (Logical Data Independence):** Menyediakan antarmuka (interface) data yang stabil. Jika skema fisik tabel dasar diubah (misalnya normalisasi ulang, pemecahan tabel, atau penggantian nama kolom fisik), antarmuka kolom View dapat tetap dipertahankan dengan alias, sehingga program aplikasi klien tidak mengalami kerusakan (breaking changes).
- **Personalisasi Akses Berbasis Peran:** Menyajikan representasi data yang disesuaikan dengan kebutuhan setiap peran: Bagian Gudang hanya melihat stok persediaan barang, Bagian Kasir melihat katalog harga aktif, dan Bagian Manajemen Keuangan melihat rekapitulasi omset per periode.

2. SINTAKS LENGKAP & MANAJEMEN VIEW DI MYSQL

MySQL menyediakan struktur sintaks deklaratif yang sangat fleksibel untuk mendefinisikan dan mengelola View:

```
CREATE
  [OR REPLACE]
  [ALGORITHM = {UNDEFINED | MERGE | TEMPTABLE}]
  [DEFINER = user]
  [SQL SECURITY { DEFINER | INVOKER }]
  VIEW view_name [(column_list)]
  AS select_statement
  [WITH [CASCADED | LOCAL] CHECK OPTION];
```

2.1 Penjelasan Parameter dan Klausa

- **OR REPLACE:** Mengganti definisi View secara otomatis apabila nama View bersangkutan telah ada di skema, tanpa perlu menjalankan DROP VIEW terlebih dahulu.
- **ALGORITHM:** Menentukan strategi eksekusi yang diterapkan oleh Query Optimizer MySQL saat View dipanggil (dibahas khusus pada Bagian 6).
- **DEFINER:** Menentukan akun pengguna MySQL pemilik/pembuat View (format default: 'user'@'host' yang mengeksekusi DDL).
- **SQL SECURITY:** Menentukan konteks hak akses keamanan saat View dieksekusi—apakah menggunakan hak milik akun pembuat (DEFINER) atau hak akun yang memanggil (INVOKER).
- **column_list:** Daftar nama kolom alias eksplisit. Jika diabaikan, View akan mewarisi nama kolom/ekspresi dari select_statement.
- **AS select_statement:** Kueri SELECT lengkap yang mendefinisikan hasil data virtual.
- **WITH CHECK OPTION:** Klausul pengaman yang membatasi operasi INSERT dan UPDATE agar selalu mematuhi kondisi WHERE pada View.

2.2 Manajemen Siklus Hidup dan Metadata View

```
-- 1. Menampilkan kueri DDL pendefinisian view yang tersimpan di kamus data
SHOW CREATE VIEW nama_view;

-- 2. Mengubah kueri definisi view tanpa merusak permission hak akses
ALTER VIEW nama_view AS
SELECT kolom1, kolom2 FROM tabel_dasar WHERE kondisi;

-- 3. Menghapus view secara aman
DROP VIEW IF EXISTS nama_view;

-- 4. Mengubah nama view
RENAME TABLE view_lama TO view_baru;

-- 5. Memeriksa status operasional dan metadata di INFORMATION_SCHEMA
SELECT TABLE_NAME, VIEW_DEFINITION, CHECK_OPTION, IS_UPDATABLE, DEFINER, SECURITY_TYPE
FROM INFORMATION_SCHEMA.VIEWS
WHERE TABLE_SCHEMA = DATABASE();
```

3. KLASIFIKASI VIEW: SIMPLE VIEW VS COMPLEX VIEW

Berdasarkan struktur kueri pembentuknya, View terbagi menjadi dua klasifikasi utama dengan karakteristik yang sangat berbeda:

Parameter Karakteristik	Simple View	Complex View
Jumlah Tabel Sumber	Hanya berasal dari 1 tabel fisik dasar (single table).	Melibatkan 2 tabel atau lebih menggunakan JOIN.
Fungsi Agregasi	Tidak mengandung fungsi agregat (SUM, AVG, COUNT, dll).	Umumnya mengandung fungsi agregat dan analisis nilai.
Pengelompokan Data	Tidak menggunakan klausul GROUP BY atau HAVING.	Menggunakan klausul GROUP BY dan filter HAVING.

Eliminasi Duplikasi	Tidak menggunakan klausul DISTINCT.	Dapat menggunakan klausa DISTINCT atau UNION.
Proyeksi Kolom	Kolom memetakan 1-ke-1 langsung ke kolom tabel fisik.	Mengandung ekspresi kalkulasi (misal: banyak * hargajual).
Operasi DML (Update/Insert)	Dapat di-UPDATE, di-INSERT, dan di-DELETE (Updatable).	Sebagian besar bersifat Read-Only (Hanya Baca).

4. UPDATABLE VIEWS (MEMPERBARUI DATA MELALUI VIEW)

Updatable View adalah View yang memungkinkan eksekusi operasi DML (INSERT, UPDATE, DELETE), di mana sistem RDBMS akan secara otomatis meneruskan mutasi data tersebut ke tabel fisik dasar di bawahnya.

4.1 Syarat dan Batasan Mutlak MySQL untuk Updatable View

MySQL menerapkan aturan ketat. Sebuah View menjadi Non-Updatable (Read-Only) apabila definisi kuerinya mengandung salah satu dari konstruksi berikut:

- **1.** Fungsi Agregat: Penggunaan fungsi SUM(), MIN(), MAX(), COUNT(), AVG(), atau variasinya.
- **2.** Klausa DISTINCT: Menghilangkan korespondensi baris 1-ke-1 antara baris View dan baris fisik tabel dasar.
- **3.** Klausa GROUP BY atau HAVING: Pengelompokan baris menghasilkan ringkasan data, sehingga engine tidak dapat mengidentifikasi baris fisik mana yang harus dimodifikasi.
- **4.** Operator Himpunan: Penggunaan UNION atau UNION ALL.
- **5.** Kolom Ekspresi atau Kalkulasi: Kolom yang dihasilkan dari perhitungan aritmatika (misal: banyak * hargajual) atau manipulasi teks tidak dapat di-UPDATE.
- **6.** Subquery: Terdapat subquery di dalam SELECT-list atau klausa FROM yang merujuk ke tabel target pembaruan.
- **7.** Algoritma TEMPTABLE: View yang dieksekusi melalui temporary table selalu bersifat read-only.
- **8.** Batasan Kolom NOT NULL pada INSERT: Untuk melakukan operasi INSERT melalui View, seluruh kolom pada tabel fisik dasar yang berstatus NOT NULL dan tidak memiliki nilai DEFAULT wajib dimasukkan dalam daftar proyeksi kolom View.

4.2 Aturan Khusus Updatable View Multi-Tabel

MySQL mengizinkan operasi UPDATE pada View yang dibentuk dari beberapa tabel (JOIN) dengan ketentuan ketat: perintah UPDATE hanya boleh mengubah kolom-kolom yang berasal dari SATU TABEL FISIK saja dalam satu kali eksekusi kueri. Upaya memperbarui kolom dari dua tabel fisik berbeda sekaligus dalam satu pernyataan akan ditolak oleh database engine.

5. KLAUSA WITH CHECK OPTION & INTEGRITAS DATA

Risiko utama pada Updatable View adalah anomali 'Phantom Update' (hilangnya baris data dari View). Tanpa klausa pelindung, pengguna dapat melakukan INSERT atau UPDATE data dengan nilai yang bertentangan dengan kondisi WHERE View. Data tersebut akan berhasil disimpan ke tabel fisik dasar, namun seketika menghilang dari hasil tampilan View.

5.1 Studi Kasus: Anomali Tanpa CHECK OPTION

```
-- View didefinisikan untuk produk berharga >= 5000 tanpa CHECK OPTION:
CREATE VIEW vw_ProdukMahal AS
SELECT kodeproduk, namaproduk, harga, stok
FROM produk
WHERE harga >= 5000;

-- Pengguna melakukan update data melalui view:
UPDATE vw_ProdukMahal SET harga = 2000 WHERE kodeproduk = 'P003';
```

```
-- HASIL: Kueri berhasil dijalankan! Namun harga produk 'P003' sekarang 2000,
-- sehingga baris produk tersebut seketika hilang dari tampilan 'vw_ProdukMahal'.
```

5.2 Mekanisme Kerja WITH CHECK OPTION

Dengan menambahkan klausa WITH CHECK OPTION pada akhir deklarasi View, MySQL akan secara aktif memvalidasi setiap baris yang di-INSERT atau di-UPDATE. Jika nilai baru menyebabkan kondisi WHERE bernilai FALSE, operasi langsung dibatalkan secara atomik dan menghasilkan galat:

```
ERROR 1369 (HY000): CHECK OPTION failed 'minimarket.vw_ProdukMahal'
```

5.3 Analisis Komparasi: LOCAL vs CASCADED CHECK OPTION

Pada skenario View bersarang (Nested View / View yang merujuk View lain), MySQL menyediakan dua cakupan validasi:

Klausul	Cakupan Validasi Integritas	Evaluasi terhadap Underlying View
LOCAL CHECK OPTION	Hanya menguji kondisi WHERE dari View bersangkutan.	Kondisi WHERE dari View di bawahnya HANYA diuji jika View di bawahnya tersebut secara eksplisit memiliki klausa CHECK OPTION.
CASCADED CHECK OPTION (Default)	Menguji kondisi WHERE dari View bersangkutan DAN seluruh rantai View di bawahnya.	Kondisi WHERE seluruh View di bawahnya akan tetap diuji secara rekursif, meskipun View di bawahnya TIDAK mendeklarasikan CHECK OPTION.

6. ALGORITMA PEMROSESAN VIEW (VIEW PROCESSING ALGORITHMS)

MySQL menyediakan tiga pilihan algoritma eksekusi kueri pada klausa ALGORITHM:

- **ALGORITHM = MERGE:** Mekanisme: Kueri pemanggil akan digabungkan secara langsung (rewritten) dengan definisi kueri View menjadi satu kueri terpadu sebelum dioptimasi dan dieksekusi. Kelebihan: Sangat efisien, mendukung pemanfaatan indeks tabel fisik dasar, dan memungkinkan Updatable View. Batasan: Hanya dapat diterapkan pada View yang memenuhi kriteria Simple View.
- **ALGORITHM = TEMPTABLE:** Mekanisme: MySQL mengeksekusi kueri View terlebih dahulu, menyimpan hasilnya ke dalam tabel sementara (temporary table) di memori/disk, kemudian mengeksekusi kueri pemanggil terhadap tabel sementara tersebut. Kelebihan: Mampu menangani kueri kompleks dengan agregasi, GROUP BY, dan UNION. Kelemahan: Mengonsumsi memori tambahan, indeks tabel dasar tidak dapat dimanfaatkan oleh kueri luar, dan View selalu bersifat Read-Only.
- **ALGORITHM = UNDEFINED:** Mekanisme: Nilai bawaan (default). MySQL Optimizer secara otomatis memilih MERGE jika memungkinkan demi performa optimal, dan otomatis beralih ke TEMPTABLE jika kueri mengandung fungsi yang melarang MERGE.

7. KEAMANAN & HAK AKSES VIEW (VIEW SECURITY & PRIVILEGES)

View bertindak sebagai gerbang keamanan (security barrier) yang membatasi interaksi pengguna dengan tabel dasar. Klausul SQL SECURITY menentukan hak akses mana yang dievaluasi saat View dipanggil:

- **SQL SECURITY DEFINER (Default):** View dieksekusi dengan hak akses milik pembuat View (Definer). Pengguna yang memanggil View hanya membutuhkan izin SELECT pada View tersebut dan TIDAK PERLU memiliki izin akses ke tabel

fisik dasar. Skenario: Digunakan untuk memberikan hak pembacaan data transaksi teragregasi kepada kasir tanpa memberi akses langsung ke tabel NOTA.

- **SQL SECURITY INVOKER:** View dieksekusi dengan hak akses milik pengguna yang sedang menjalankan kueri (Invoker). Pengguna harus memiliki hak akses pada View DAN sekaligus memiliki hak akses pada semua tabel fisik yang mendasarinya. Skenario: Digunakan saat otorisasi data harus dikontrol ketat oleh hak akses pengguna individual.

8. BEST PRACTICES & LIMITASI VIEW DI MYSQL

8.1 Praktik Terbaik Desain View

- **1. Proyeksi Kolom Eksplisit:** Gunakan proyeksi kolom eksplisit. Hindari penggunaan 'SELECT *' dalam pendefinisian View produksi agar penambahan kolom di tabel dasar tidak merusak struktur View.
- **2. Hindari Deep Nesting:** Batasi kedalaman View bersarang (View di atas View) maksimal 2 hingga 3 tingkat guna mencegah penurunan performa eksekusi kueri.
- **3. Pengindeksan Kolom Kunci:** Pastikan kolom-kolom yang menjadi predikat WHERE dan JOIN pada definisi View telah memiliki indeks pada tabel fisik dasarnya.
- **4. Konvensi Penamaan Terstandar:** Terapkan standar penamaan konsisten dengan awalan seperti 'vw_' untuk memudahkan identifikasi objek virtual dalam skema basis data.

8.2 Limitasi View pada MySQL Server

- **Ketiadaan Indeks Mandiri:** View di MySQL tidak dapat memiliki indeks fisik tersendiri (berbeda dengan Indexed View pada Microsoft SQL Server).
- **Tidak Ada Materialized View Bawaan:** MySQL tidak mendukung Materialized View bawaan (penyimpanan hasil kueri secara fisik yang diperbarui berkala).
- **Larangan Trigger pada View:** MySQL tidak mengizinkan pemicu (TRIGGER) dipasang secara langsung pada objek View.

9. STUDI KASUS: SISTEM BASIS DATA TRANSAKSI MINIMARKET (DISKUSI KELOMPOK 7)

Studi kasus ini didasarkan pada hasil perancangan dan normalisasi 3NF dari Diskusi Kelompok 7 (Modul Praktikum 2). Sistem mencatat aktivitas transaksi kasir di toko ritel modern yang melayani customer umum maupun member. Skema terdiri dari 5 tabel relasional yang telah dinormalisasi secara penuh:

9.1 Rangkuman Skema Fisik & Kamus Data

a. Tabel 1: PRODUK (Master Data Barang Dagangan)

Nama Field	Tipe Data	Batasan Integritas (Constraint)	Keterangan
KodeProduk	CHAR(4)	PRIMARY KEY, NOT NULL	Kode unik barang (contoh: 'P001')
NamaProduk	VARCHAR(50)	NOT NULL	Nama deskripsi barang dagangan
Harga	INT	CHECK (Harga >= 0), NOT NULL	Harga satuan resmi barang
Stok	INT	CHECK (Stok >= 0), DEFAULT 0, NOT NULL	Jumlah unit persediaan barang di toko

b. Tabel 2: KASIR (Master Data Petugas Kasir)

Nama Field	Tipe Data	Batasan Integritas	Keterangan

		(Constraint)	
KodeKasir	CHAR(3)	PRIMARY KEY, NOT NULL	Kode unik petugas kasir (contoh: '001')
NamaKasir	VARCHAR(50)	NOT NULL	Nama lengkap petugas kasir
Pass	CHAR(6)	CHECK (CHAR_LENGTH(Pass)=6), NOT NULL	Kata sandi otentikasi (harus tepat 6 karakter)

c. Tabel 3: CUSTOMER (Master Data Pelanggan / Member)

Nama Field	Tipe Data	Batasan Integritas (Constraint)	Keterangan
KodeCus	CHAR(5)	PRIMARY KEY, NOT NULL	ID unik customer/member (contoh: 'M0001')
NamaCus	VARCHAR(50)	NOT NULL	Nama lengkap customer
TglLahir	DATE	NOT NULL	Tanggal lahir untuk program promo ulang tahun
JenKelCus	CHAR(1)	CHECK (JenKelCus IN ('L','P')), NOT NULL	Jenis kelamin: 'L' (Laki-laki) atau 'P' (Perempuan)
AlmtCus	VARCHAR(100)	DEFAULT 'Yogyakarta', NOT NULL	Alamat domisili customer
HPCus	VARCHAR(15)	NOT NULL	Nomor kontak telepon seluler
PointTerkumpul	INT	CHECK (PointTerkumpul >= 0), DEFAULT 0	Akumulasi point reward belanja customer

d. Tabel 4: NOTA (Header Transaksi Penjualan)

Nama Field	Tipe Data	Batasan Integritas (Constraint)	Keterangan
Nonota	CHAR(10)	PRIMARY KEY, NOT NULL	Nomor nota transaksi (contoh: '2208030001')
KodeKasir	CHAR(3)	FOREIGN KEY ke KASIR(KodeKasir), NOT NULL	Petugas kasir yang mencatat transaksi
TglTran	DATE	NOT NULL	Tanggal terjadinya transaksi penjualan
KodeCus	CHAR(5)	FOREIGN KEY ke CUSTOMER(KodeCus), NOT NULL	Identitas customer pembeli

e. Tabel 5: ITEMNOTA (Rincian Detil Barang Transaksi / Pivot)

Nama Field	Tipe Data	Batasan Integritas (Constraint)	Keterangan

Nonota	CHAR(10)	COMPOSITE PK, FK ke NOTA(Nonota)	Nomor nota transaksi induk
KodeProduk	CHAR(4)	COMPOSITE PK, FK ke PRODUK(KodeProduk)	Kode barang yang dibeli
Banyak	INT	CHECK (Banyak >= 1), DEFAULT 1, NOT NULL	Jumlah kuantitas unit produk yang dibeli
HargaJual	INT	CHECK (HargaJual >= 0), NOT NULL	Harga riil transaksi per unit saat nota dicetak

10. LEMBAR TUGAS & LATIHAN SOAL PRAKTIKUM (UNTUK MAHASISWA)

Petunjuk Pengerjaan Mahasiswa / Praktikan:
1. Pastikan basis data minimarket hasil praktikum sebelumnya telah aktif pada sistem MySQL Anda.
2. Tuliskan kueri SQL pembuatan View (DDL) serta kueri pengujian (DML) sesuai spesifikasi yang diminta pada tiap nomor soal.
3. Tangkap layar (screenshot) kode kueri dan hasil eksekusi program di MySQL Command Line Client / Workbench.
4. Perhatikan syarat Updatable View dan batasan integritas WITH CHECK OPTION yang telah dipelajari pada bagian materi.

Tingkat 1: View Pelaporan Sederhana & Masking Keamanan Data

Soal 1 (Katalog Display & Evaluasi Stok): Buatlah View dengan nama 'vw_KatalogProdukAktif' untuk menyajikan katalog barang kepada staf display toko. View harus menampilkan: Kode Produk, Nama Produk (ditampilkan dengan huruf kapital semua), Harga Produk, Stok Barang, dan kolom kalkulasi 'Status Stok' dengan ketentuan: jika stok = 0 maka berlabel 'Stok Habis', jika stok kurang dari 10 maka berlabel 'Stok Kritis', selain itu berlabel 'Stok Aman'. Uji dengan menampilkan data terurut berdasarkan nama produk secara alfabetis.

Soal 2 (Masking Kredensial Kasir): Demi mematuhi regulasi privasi data dan keamanan sistem, kata sandi kasir tidak boleh diekspos sembarangan. Buatlah View dengan nama 'vw_ProfilKasirPublik' yang menampilkan: Kode Kasir, Nama Kasir, dan kolom audit 'Status Kredensial' yang berisi teks 'Memenuhi Standar Keamanan' jika panjang kata sandi tepat 6 karakter, dan 'Tidak Aman' jika sebaliknya. Pastikan kolom 'Pass' TIDAK DITAMPILKAN sama sekali dalam definisi View.

Soal 3 (Masking Informasi Member Domisili): Bagian pemasaran ingin memberikan promo khusus member lokal. Buatlah View bernama 'vw_MemberYogyakarta' yang menyaring customer yang berdomisili di 'Yogyakarta'. Tampilkan: Kode Customer, Nama Customer, Keterangan Jenis Kelamin ('Laki-laki' jika 'L', 'Perempuan' jika 'P'), Alamat, dan Nomor HP dengan sensor privasi (ganti 4 digit terakhir nomor HP dengan tanda 'XXXX', contoh: '08162992' menjadi '0816XXXX').

Tingkat 2: View Analitik Transaksi & Multi-Table Joins

Soal 4 (Detail Penjualan Multi-Tabel): Buatlah View dengan nama 'vw_RincianTransaksiLengkap' yang menghubungkan kelima tabel fisik (NOTA, KASIR, CUSTOMER, ITEMNOTA, PRODUK). Kolom yang ditampilkan meliputi: Nomor Nota, Tanggal Transaksi, Nama Kasir yang melayani, Nama Customer, Nama Produk yang dibeli, Banyak (kuantitas), Harga Jual riil, dan kolom hitung 'Subtotal' (hasil kali Banyak dengan Harga Jual). Uji dengan menampilkan transaksi yang terjadi di awal bulan Agustus 2022.

Soal 5 (Rekapitulasi Faktur & Omset Transaksi): Pihak manajemen keuangan membutuhkan ringkasan omset per transaksi. Buatlah View bernama 'vw_RekapNotaPenjualan' yang menyajikan informasi tingkat nota: Nomor Nota, Tanggal Transaksi, Nama Kasir, Nama Customer, Jumlah Variasi Item Barang yang Dibeli (hitung item unik), Total Kuantitas Unit Barang yang Dibeli (akumulasi banyak), dan 'Total Belanja Nota' (total akumulasi subtotal rupiah). Gunakan pengelompokan data yang tepat.

Soal 6 (Analitik Kinerja Produktivitas Kasir): Buatlah View dengan nama 'vw_KinerjaKasir' untuk evaluasi produktivitas bulanan. Tampilkan: Kode Kasir, Nama Kasir, Jumlah Transaksi Nota yang Berhasil Dicatat (frekuensi nota unik), Total Nominal Omset Penjualan yang Berhasil Dihasilkan oleh kasir tersebut, serta Rata-rata Nominal per Nota (nilai rata-rata belanja konsumen yang dilayani). Urutkan dari kasir dengan total omset tertinggi.

Tingkat 3: Updatable View & Manajemen Data Operasional

Soal 7 (Pembaruan Stok Barang via View): Staf gudang diberikan kewenangan untuk memperbarui stok persediaan barang tanpa diizinkan melihat atau mengubah harga master. a) Buatlah View sederhana bernama 'vw_UpdateStokGudang' yang hanya memproyeksikan: KodeProduk, NamaProduk, dan Stok dari tabel PRODUK.
b) Tuliskan kueri UPDATE melalui 'vw_UpdateStokGudang' untuk mengubah stok produk 'P001' menjadi 25 unit.
c) Buktikan dengan kueri SELECT bahwa data pada tabel fisik dasar (PRODUK) ikut berubah secara otomatis.

Soal 8 (Pendaftaran Member Baru Tanpa Field Point): Bagian pendaftaran member baru hanya diizinkan menginput identitas customer dasar tanpa hak mengisi saldo point reward. a) Buatlah View 'vw_RegistrasiCustomer' yang mencakup

kolom: KodeCus, NamaCus, TglLahir, JenKelCus, AlmtCus, dan HPCus.
b) Lakukan operasi INSERT data satu customer baru melalui View tersebut:
('M0003', 'Siti Rahma', '1995-04-12', 'P', 'Yogyakarta', '081333445566').
c) Periksa tabel fisik CUSTOMER dan jelaskan nilai kolom PointTerkumpul yang otomatis terbentuk beserta alasannya.

Tingkat 4: Integritas Data & Proteksi dengan WITH CHECK OPTION

Soal 9 (Validasi Batasan Nilai Harga Produk): Toko ingin membuat daftar kelola produk premium (harga >= 5000) yang harus selalu terjamin kepatuhan harganya. a) Buatlah View bernama 'vw_ProdukPremium' yang memilih KodeProduk, NamaProduk, Harga, dan Stok dengan syarat harga >= 5000 dan sematkan klausa 'WITH CHECK OPTION'.
b) Uji coba 1: Lakukan penambahan produk baru ('P004', 'Susu Steril Premium', 9500, 10) melalui View tersebut. Amati hasilnya.
c) Uji coba 2: Lakukan kueri UPDATE untuk menurunkan harga produk 'P004' menjadi 3000 melalui View. Catat pesan respon/galat yang dimunculkan oleh MySQL dan berikan analisis teknisnya.

Soal 10 (Eksperimen Tingkat Lanjut: LOCAL vs CASCADED): Toko menerapkan skema segmentasi pelanggan bertingkat (nested view):
- View Induk: 'vw_MemberYogya' memilih seluruh customer yang beralamat di 'Yogyakarta'.
- View Anak: 'vw_MemberLoyalYogya' diturunkan dari 'vw_MemberYogya' dengan filter tambahan 'PointTerkumpul >= 20'.
Tugas Praktikan:
a) Implementasikan kedua View tersebut dengan menyematkan klausa 'WITH LOCAL CHECK OPTION' pada View Anak.
b) Lakukan pengujian pembaruan alamat customer menjadi 'Sleman' melalui View Anak. Apakah operasi tersebut berhasil atau gagal?
c) Gantilah klausa View Anak menjadi 'WITH CASCADED CHECK OPTION', lalu ulangi pengujian yang sama. Analisis perbedaan perilakunya!

11. LAMPIRAN: KUNCI JAWABAN & PEMBAHASAN LENGKAP (KHUSUS DOSEN & ASISTEN PRAKTIKUM)

Bagian ini disediakan khusus untuk instruktur dan asisten praktikum sebagai pedoman penilaian, memuat solusi kueri SQL baku, skrip pengujian operasional, dan analisis teknis keluaran database engine:

Solusi & Pembahasan Soal 1 (vw_KatalogProdukAktif)

```
-- DDL Pembuatan View
CREATE OR REPLACE VIEW vw_KatalogProdukAktif AS
SELECT
   kodeproduk,
   UPPER(namaproduk) AS nama_produk_kapital,
   harga,
   stok,
   CASE
      WHEN stok = 0 THEN 'Stok Habis'
      WHEN stok < 10 THEN 'Stok Kritis'
      ELSE 'Stok Aman'
   END AS status_stok
FROM produk;

-- Kueri Pengujian
SELECT * FROM vw_KatalogProdukAktif ORDER BY nama_produk_kapital ASC;
```

Analisis Logika: Kueri memanfaatkan fungsi string bawaan UPPER() untuk menstandarkan tampilan nama barang dan struktur kendali CASE-WHEN-THEN-ELSE-END untuk mengevaluasi ambang batas unit persediaan. View ini berstatus read-only untuk kolom kalkulasi status_stok, namun jika dialgoritmakan MERGE, kolom harga dan stok tabel aslinya tetap dapat diakses.

Solusi & Pembahasan Soal 2 (vw_ProfilKasirPublik)

```
-- DDL Pembuatan View
CREATE OR REPLACE VIEW vw_ProfilKasirPublik AS
SELECT
   kodekasir,
   namakasir,
   IF(CHAR_LENGTH(pass) = 6, 'Memenuhi Standar Keamanan', 'Tidak Aman') AS status_kredensial
FROM kasir;

-- Kueri Pengujian
SELECT * FROM vw_ProfilKasirPublik;
```

Analisis Logika: Penerapan teknik Proyeksi Terseleksi (Column Masking). Field 'pass' yang berisi teks kata sandi sengaja tidak diproyeksikan ke SELECT-list, melainkan hanya dievaluasi panjang karakternya menggunakan CHAR_LENGTH() yang dibungkus conditional IF(). Pengguna publik yang hanya memiliki izin SELECT pada View ini secara absolut tidak dapat membocorkan password kasir.

Solusi & Pembahasan Soal 3 (vw_MemberYogyakarta)

```
-- DDL Pembuatan View
CREATE OR REPLACE VIEW vw_MemberYogyakarta AS
SELECT
   kodecus,
```

```
    namacus,
    IF(jenkelcus = 'L', 'Laki-laki', 'Perempuan') AS jenis_kelamin,
    almtcus,
    CONCAT(LEFT(hpcus, CHAR_LENGTH(hpcus) - 4), 'XXXX') AS nomor_hp_sensor,
    pointterkumpul
FROM customer
WHERE almtcus = 'Yogyakarta';

-- Kueri Pengujian
SELECT * FROM vw_MemberYogyakarta;
```

Analisis Logika: Penggunaan fungsi manipulasi string CONCAT() dipadukan dengan LEFT() dan CHAR_LENGTH() untuk memproyeksikan seluruh digit awal nomor ponsel dan mengganti 4 digit terakhir dengan string 'XXXX'. Filter WHERE almtcus = 'Yogyakarta' memastikan data tersaring akurat.

Solusi & Pembahasan Soal 4 (vw_RincianTransaksiLengkap)

```
-- DDL Pembuatan View Multi-Table JOIN
CREATE OR REPLACE VIEW vw_RincianTransaksiLengkap AS
SELECT
    n.nonota,
    n.tgltran,
    k.namakasir,
    c.namacus,
    p.namaproduk,
    i.banyak,
    i.hargajual,
    (i.banyak * i.hargajual) AS subtotal
FROM nota n
INNER JOIN kasir k ON n.kodekasir = k.kodekasir
INNER JOIN customer c ON n.kodecus = c.kodecus
INNER JOIN itemnota i ON n.nonota = i.nonota
INNER JOIN produk p ON i.kodeproduk = p.kodeproduk;

-- Kueri Pengujian Filter Transaksi 1 s.d. 3 Agustus 2022
SELECT * FROM vw_RincianTransaksiLengkap
WHERE tgltran BETWEEN '2022-08-01' AND '2022-08-03'
ORDER BY nonota, namaproduk;
```

Analisis Logika: Mengintegrasikan data relasional 3NF dari kelima tabel master dan transaksi melalui INNER JOIN. Kueri mengabstraksikan kerumitan foreign key nota-itemnota-produk sehingga pengguna cukup melihat entitas nama deskriptif disertai hasil kali kalkulasi subtotal.

Solusi & Pembahasan Soal 5 (vw_RekapNotaPenjualan)

```
-- DDL Pembuatan View Agregasi Tingkat Faktur
CREATE OR REPLACE VIEW vw_RekapNotaPenjualan AS
SELECT
    n.nonota,
    n.tgltran,
    k.namakasir,
```

```
    c.namacus,
    COUNT(i.kodeproduk) AS ragam_item,
    SUM(i.banyak) AS total_unit_barang,
    SUM(i.banyak * i.hargajual) AS total_belanja_nota
FROM nota n
INNER JOIN kasir k ON n.kodekasir = k.kodekasir
INNER JOIN customer c ON n.kodecus = c.kodecus
INNER JOIN itemnota i ON n.nonota = i.nonota
GROUP BY n.nonota, n.tgltran, k.namakasir, c.namacus;

-- Kueri Pengujian
SELECT * FROM vw_RekapNotaPenjualan ORDER BY nonota ASC;
```

Analisis Logika: View ini menggunakan fungsi agregasi COUNT() untuk menghitung variasi produk unik per faktur dan fungsi SUM() untuk menghitung total akumulasi kuantitas fisik dan nominal belanja. Kehadiran klausul GROUP BY dan fungsi agregat mengklasifikasikan View ini sebagai Complex View yang berstatus Non-Updatable.

Solusi & Pembahasan Soal 6 (vw_KinerjaKasir)

```
-- DDL Pembuatan View Kinerja Kasir
CREATE OR REPLACE VIEW vw_KinerjaKasir AS
SELECT
    k.kodekasir,
    k.namakasir,
    COUNT(DISTINCT n.nonota) AS frekuensi_transaksi,
    IFNULL(SUM(i.banyak * i.hargajual), 0) AS total_omset_penjualan,
    IFNULL(ROUND(AVG(t.total_nota), 2), 0) AS rata_rata_per_nota
FROM kasir k
LEFT JOIN nota n ON k.kodekasir = n.kodekasir
LEFT JOIN itemnota i ON n.nonota = i.nonota
LEFT JOIN (
    SELECT nonota, SUM(banyak * hargajual) AS total_nota
    FROM itemnota
    GROUP BY nonota
) t ON n.nonota = t.nonota
GROUP BY k.kodekasir, k.namakasir;

-- Kueri Pengujian
SELECT * FROM vw_KinerjaKasir ORDER BY total_omset_penjualan DESC;
```

Analisis Logika: Menerapkan LEFT JOIN agar staf kasir yang belum mencatatkan transaksi tetap terdaftar dalam rekap. COUNT(DISTINCT n.nonota) mencegah distorsi penggandaan faktur akibat relasi 1-ke-banyak dengan tabel itemnota, dan IFNULL() menggantikan representasi kosong dengan angka 0.

Solusi & Pembahasan Soal 7 (vw_UpdateStokGudang)

```
-- DDL Pembuatan Updatable View
CREATE OR REPLACE VIEW vw_UpdateStokGudang AS
SELECT kodeproduk, namaproduk, stok
FROM produk;
```

```
-- Pengujian Eksekusi UPDATE Melalui View
UPDATE vw_UpdateStokGudang
SET stok = 25
WHERE kodeproduk = 'P001';

-- Verifikasi Perubahan pada Tabel Fisik PRODUK
SELECT * FROM produk WHERE kodeproduk = 'P001';
```

Analisis Logika: View ini memenuhi seluruh syarat Updatable View: satu tabel dasar, tanpa agregasi, tanpa DISTINCT, dan memetakan langsung ke kolom dasar. Perintah UPDATE berhasil dieksekusi dan diverifikasi nilainya bermutasi langsung pada media penyimpanan tabel fisik PRODUK.

Solusi & Pembahasan Soal 8 (vw_RegistrasiCustomer)

```
-- DDL Pembuatan View Registrasi
CREATE OR REPLACE VIEW vw_RegistrasiCustomer AS
SELECT kodecus, namacus, tgllahir, jenkelcus, almtcus, hpcus
FROM customer;

-- Eksekusi INSERT Baris Data Baru Melalui View
INSERT INTO vw_RegistrasiCustomer (kodecus, namacus, tgllahir, jenkelcus, almtcus, hpcus)
VALUES ('M0003', 'Siti Rahma', '1995-04-12', 'P', 'Yogyakarta', '081333445566');

-- Verifikasi Data pada Tabel Fisik CUSTOMER
SELECT * FROM customer WHERE kodecus = 'M0003';
```

Analisis Logika: Operasi INSERT berhasil karena seluruh kolom NOT NULL tanpa nilai default pada tabel CUSTOMER telah diproyeksikan dalam View. Kolom 'PointTerkumpul' yang dihilangkan secara otomatis diisi oleh engine MySQL dengan nilai default (0) sesuai deklarasi skema fisik tabel.

Solusi & Pembahasan Soal 9 (vw_ProdukPremium dengan WITH CHECK OPTION)

```
-- DDL Pembuatan View dengan CHECK OPTION
CREATE OR REPLACE VIEW vw_ProdukPremium AS
SELECT kodeproduk, namaproduk, harga, stok
FROM produk
WHERE harga >= 5000
WITH CHECK OPTION;

-- Uji Coba 1: Penambahan Data Memenuhi Syarat (Harga = 9500 >= 5000) -> SUKSES
INSERT INTO vw_ProdukPremium (kodeproduk, namaproduk, harga, stok)
VALUES ('P004', 'Susu Steril Premium', 9500, 10);

-- Uji Coba 2: Pembaruan Melanggar Syarat (Harga diubah menjadi 3000 < 5000) -> GAGAL
UPDATE vw_ProdukPremium
SET harga = 3000
WHERE kodeproduk = 'P004';
-- Respon Galat: ERROR 1369 (HY000): CHECK OPTION failed 'minimarket.vw_ProdukPremium'
```

Analisis Logika: Uji coba 1 berhasil karena harga 9500 >= 5000. Uji coba 2 digagalkan oleh MySQL dengan kode kesalahan ERROR 1369 (HY000). Klausa WITH CHECK OPTION secara aktif menolak setiap modifikasi data yang menyebabkan baris bersangkutan melanggar filter WHERE, sehingga integritas himpunan data View tetap terjaga utuh.

Solusi & Pembahasan Soal 10 (Eksperimen LOCAL vs CASCADED CHECK OPTION)

```
-- 1. Membuat View Induk (Tanpa Check Option)
CREATE OR REPLACE VIEW vw_MemberYogya AS
SELECT kodecus, namacus, almtcus, pointterkumpul
FROM customer
WHERE almtcus = 'Yogyakarta';

-- 2. Eksperimen Kasus A: View Anak Menggunakan LOCAL CHECK OPTION
CREATE OR REPLACE VIEW vw_MemberLoyalYogya_Local AS
SELECT kodecus, namacus, almtcus, pointterkumpul
FROM vw_MemberYogya
WHERE pointterkumpul >= 20
WITH LOCAL CHECK OPTION;

-- Uji Kasus A: Mengubah Alamat Customer Menjadi 'Sleman'
UPDATE vw_MemberLoyalYogya_Local
SET almtcus = 'Sleman'
WHERE kodecus = 'M0001';
-- HASIL KASUS A: BERHASIL! Karena LOCAL hanya menguji kondisi dirinya (point >= 20).
-- Kondisi view induk (almtcus = 'Yogyakarta') TIDAK diuji karena view induk tidak memiliki check option.

-- 3. Eksperimen Kasus B: View Anak Menggunakan CASCADED CHECK OPTION
CREATE OR REPLACE VIEW vw_MemberLoyalYogya_Cascaded AS
SELECT kodecus, namacus, almtcus, pointterkumpul
FROM vw_MemberYogya
WHERE pointterkumpul >= 20
WITH CASCADED CHECK OPTION;

-- Kembalikan alamat M0001 ke Yogyakarta:
UPDATE customer SET almtcus = 'Yogyakarta' WHERE kodecus = 'M0001';

-- Uji Kasus B: Mengubah Alamat Customer Menjadi 'Sleman' Melalui View Cascaded
UPDATE vw_MemberLoyalYogya_Cascaded
SET almtcus = 'Sleman'
WHERE kodecus = 'M0001';
-- HASIL KASUS B: DITOLAK! ERROR 1369 (HY000): CHECK OPTION failed.
-- Karena CASCADED secara rekursif menguji kondisi view dirinya DAN seluruh rantai view di bawahnya.
```

Analisis Logika: Eksperimen ini membuktikan perbedaan hakiki LOCAL vs CASCADED: Pada LOCAL CHECK OPTION, MySQL hanya menguji kondisi dirinya (pointterkumpul >= 20) dan mengabaikan view induk yang tidak ber-check option. Sedangkan pada CASCADED, MySQL mewajibkan kepatuhan terhadap seluruh rantai kondisi view di bawahnya secara rekursif.

12. DAFTAR PUSTAKA & REFERENSI RESMI

- **1.** Oracle Corporation. (2024). MySQL 8.0 Reference Manual: CREATE VIEW Statement & Updatable Views. https://dev.mysql.com/doc/refman/8.0/en/create-view.html

- **2.** Carlos Coronel & Steven Morris. (2019). Database Systems: Design, Implementation, & Management (13th Edition). Cengage Learning, Boston, USA.
- **3.** Toby J. Teorey, Sam S. Lightstone, Tom Nadeau, H.V. Jagadish. (2011). Database Modeling and Design: Logical Design (5th Edition). Elsevier / Morgan Kaufmann.
- **4.** Laboratorium Sistem Informasi. (2024). Modul Praktikum Sistem Manajemen Basis Data Lanjut: Materi Praktikum 1 s.d. 4. Universitas Amikom Yogyakarta.